

AWS Lambda – Implementation

Project Title

Implementation of AWS Lambda for Serverless Computing

Objective

The objective of this implementation is to understand AWS Lambda and demonstrate how serverless code can be created, deployed, executed, and monitored without managing physical or virtual servers.

Introduction

AWS Lambda is a serverless compute service provided by AWS. It allows users to run application code in response to events without provisioning or managing servers.

Lambda automatically manages the underlying compute infrastructure and can scale the application based on incoming requests.

Common use cases include:

- Processing files uploaded to Amazon S3.
- Running backend APIs.
- Processing data.
- Automating AWS operations.
- Sending notifications.
- Running scheduled tasks.
- Processing application events.

Implementation Steps

Step 1: Open AWS Lambda

Navigate to:

AWS Management Console → Lambda → Functions

Click **Create function**.

Step 2: Create Function

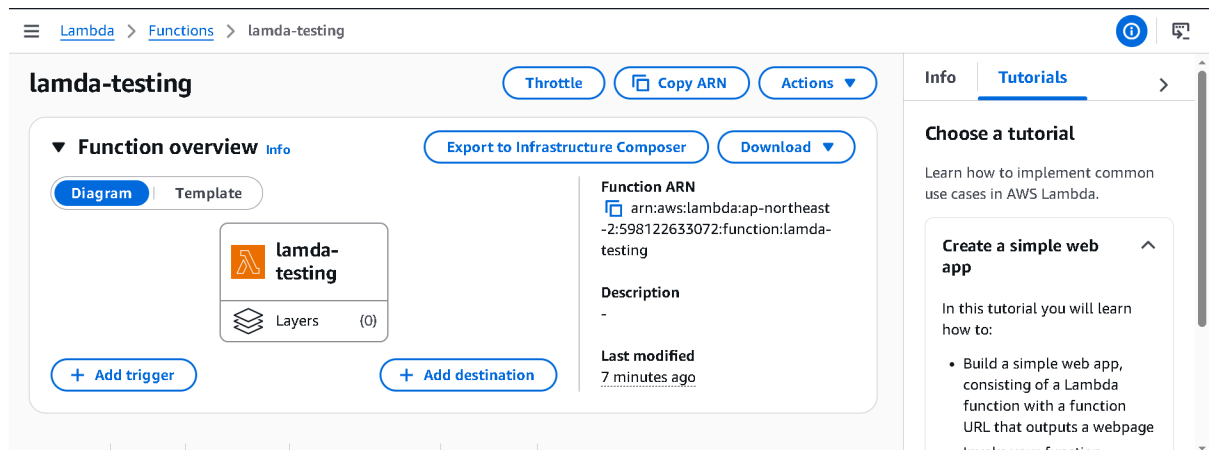
Select:

Create the function.

Click **Deploy** after entering the code.

Step 4: Test the Function

A test event was created from the Lambda console and the function was executed



Lambda → Monitor → View CloudWatch logs

Screenshot

Observations

- Lambda successfully executed the Python function.
- No EC2 instance or server was required.
- The function executed only when invoked.
- Lambda automatically manages the underlying infrastructure.
- Execution logs were available through CloudWatch.
- Lambda is suitable for event-driven and serverless applications.

Advantages

- No server management.
- Automatic scaling.
- Event-driven execution.
- Easy integration with AWS services.
- Pay-per-use pricing model.
- Built-in monitoring through CloudWatch

2. AWS API Gateway – Implementation

Project Title

Implementation of Amazon API Gateway for Creating and Managing APIs

Objective

The objective of this implementation is to understand Amazon API Gateway and demonstrate how an HTTP API can be created and connected to a backend service.

Introduction

Amazon API Gateway is a fully managed AWS service used to create, publish, maintain, monitor, and secure APIs.

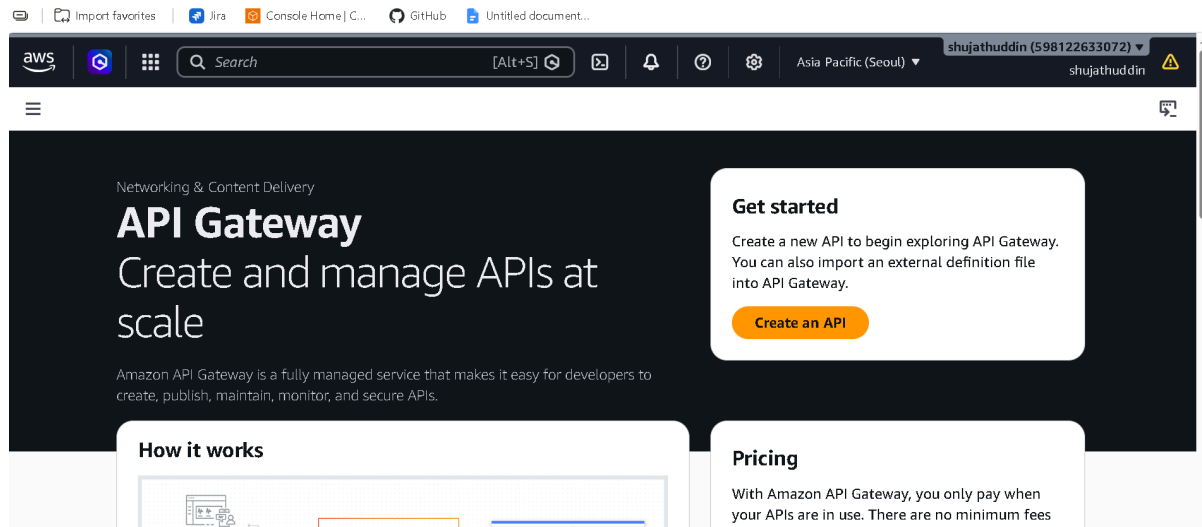
It acts as an entry point through which applications can communicate with backend services.

API Gateway can be used with services such as:

- AWS Lambda
- EC2 applications
- Other AWS services
- External backend applications

Common use cases include:

- REST APIs
- HTTP APIs
- Mobile application backends
- Web application backends
- Microservices
- Serverless applications



API Gateway Features

API Gateway provides features such as:

- API routing
- HTTP/HTTPS communication
- Authentication and authorization options
- Throttling
- Monitoring
- API lifecycle management
- Integration with backend services

Observations

- An HTTP API was successfully created.
- A backend integration was configured.
- An API route was created.
- The API endpoint successfully accepted HTTP requests.
- The backend service processed the request.
- The API response was successfully returned to the client.

Advantages

- Fully managed API service.
- Easy integration with AWS services.
- Supports HTTPS.
- Provides API management capabilities.
- Supports authentication and authorization mechanisms.
- Useful for serverless and microservice architectures.

3. AWS KMS – Implementation

Project Title

Implementation of **AWS Key Management Service (KMS)** for Data Encryption

Objective

The objective of this implementation is to understand AWS Key Management Service and demonstrate how encryption keys can be created and managed for protecting data.

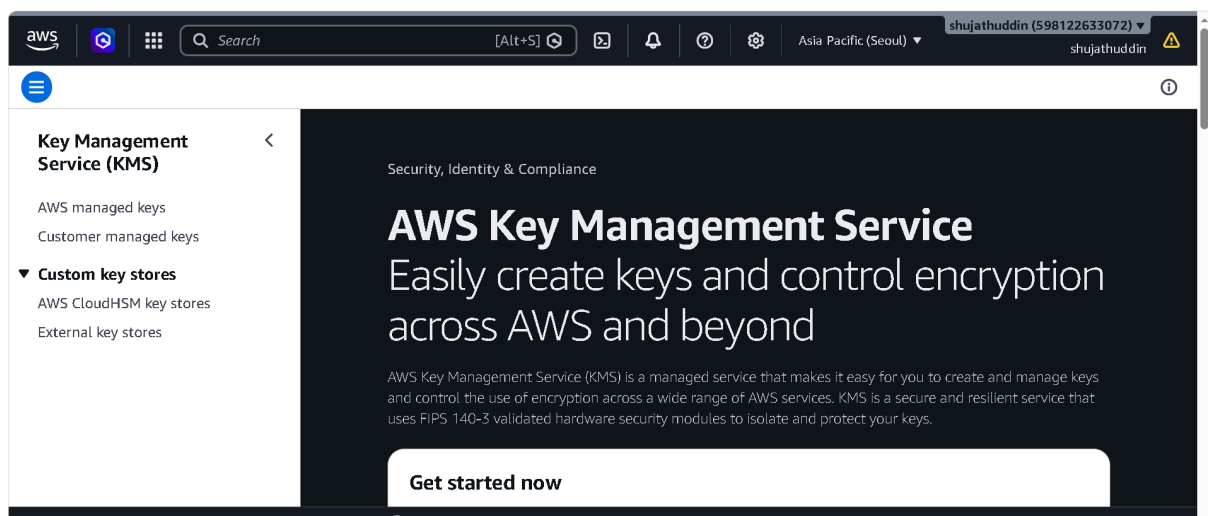
Introduction

AWS Key Management Service (AWS KMS) is a managed AWS service used to create and control cryptographic keys.

KMS helps protect data stored in AWS services by providing encryption keys and controlling access to cryptographic operations.

KMS can be integrated with services such as:

- Amazon S3
- Amazon EBS
- Amazon RDS
- Amazon DynamoDB
- AWS Secrets Manager
- Other AWS services



Observations

- A customer-managed symmetric KMS key was successfully created.
- An alias was configured for easier identification.
- Key administrators and key users can be controlled through permissions.
- KMS provides centralized management of encryption keys.
- KMS can be integrated with multiple AWS services.
- Encryption helps protect sensitive data at rest.

Advantages

- Centralized key management.
- Strong integration with AWS services.
- Access control through IAM and key policies.
- Supports encryption and decryption.
- Provides auditing capabilities through AWS CloudTrail.
- Helps protect sensitive data.